

Comparative Analysis of Clustering Algorithms: K-Means, DBSCAN, and Gaussian Mixture Model

Peter Yacoub, Mohamed Malek Kaouach, Soud Asaad Soud Alhazba, and Mohammad Azmi Al-Betar

Artificial Intelligence Research Center (AIRC)

College of Engineering and Information Technology

Ajman University, Ajman, UAE

peteryaqoob1996@hotmail.com, 202510024@ajmanuni.ac.ae, 202420468@ajmanuni.ac.ae, m.albetar@ajman.ac.ae

Abstract—Clustering is a fundamental task in unsupervised machine learning, yet the choice of algorithm and its hyperparameters strongly influences the discovered data structure. This paper presents a comparative study of three widely used clustering methods: K-Means, DBSCAN, and Gaussian Mixture Models (GMMs), applied to nine benchmark datasets with varying sizes, dimensionalities, and cluster complexities. All algorithms are implemented under a unified experimental pipeline that includes duplicate removal, outlier filtering, feature standardization, and PCA-based visualization. For each dataset, algorithm-specific parameters are tuned in a controlled manner. Quantitative performance is assessed using external ground-truth labels through purity, accuracy, macro-precision, macro-recall, and macro F1-score, with additional discussion of noise handling and cluster-shape flexibility. The results show that K-Means is efficient and competitive on well-separated, approximately spherical clusters; GMMs excel on datasets with overlapping or elliptical structures; and DBSCAN is most robust to noise and non-convex cluster shapes, at the cost of sensitivity to parameter selection and reduced coverage when many points are labeled as noise. The study concludes with practical guidelines for selecting clustering algorithms and configuring their parameters in data mining applications with heterogeneous data characteristics. The results contribute a standardized benchmarking framework for unsupervised models, highlighting critical trade-offs between geometric assumptions and noise handling. This provides a decision-making roadmap for practitioners based on empirical data characteristics rather than heuristic selection.

Index Terms—clustering techniques, K-Means, DBSCAN, Gaussian Mixture Models, data mining

I. INTRODUCTION

A. General background about clustering

Clustering is a core task in unsupervised machine learning that groups unlabeled data points into clusters such that points within the same cluster are more similar to each other than to those in other clusters [6]. Unlike supervised learning, clustering does not use predefined class labels and is mainly applied to exploratory data analysis, knowledge discovery, and revealing hidden structure in complex datasets. Its applications include customer segmentation, anomaly detection, document categorization, image analysis, and bioinformatics [6], [7].

In data mining, clustering helps reduce data complexity and summarize large datasets in a compact and interpretable form. It is also commonly used as a pre-processing step for tasks such as classification, visualization, and feature engineering. However, clustering performance depends strongly on the

choice of distance measure, data scaling, noise, outliers, and the algorithm's assumptions about cluster shape and distribution [6], [7]. For this reason, comparative evaluation under diverse conditions is important for understanding the practical strengths and limitations of clustering methods.

B. General clustering methods

Clustering algorithms are commonly grouped into major paradigms based on how they define clusters [6], [7]. Partition-based methods (e.g., K-Means) optimize compactness around cluster prototypes, density-based methods (e.g., DBSCAN) identify dense regions separated by sparse areas [8], [9], and model-based methods (e.g., GMM) assume data are generated by mixtures of probability distributions [2], [4]. This work focuses on these three paradigms as they cover complementary assumptions about cluster geometry, overlap, and noise.

C. Problem statement

Despite extensive theoretical and empirical research, selecting an appropriate clustering algorithm for a given dataset remains a practical challenge. Different algorithms rely on different assumptions about cluster shape, density, and noise, and their performance can vary significantly across datasets with different characteristics [6], [7]. Centroid-based methods may fail on non-convex or imbalanced clusters, density-based methods require careful parameter tuning and may struggle in high dimensions, and model-based methods depend on distributional assumptions and component selection [5], [8], [9].

Although prior studies have discussed the strengths and limitations of centroid-based, density-based, and model-based clustering methods [6], [7], and benchmarking work has shown that performance depends strongly on dataset properties such as cluster geometry, overlap, and noise [5], many existing comparisons focus on a limited set of algorithms or datasets, or lack a unified preprocessing and evaluation pipeline. Therefore, a systematic comparison of representative clustering algorithms under consistent preprocessing, tuning, and evaluation settings is needed. This work addresses that gap by comparing K-Means, DBSCAN, and GMM on nine benchmark datasets with different sizes, dimensionalities, and cluster complexities, helping bridge the gap between theoretical assumptions and practical deployment.

D. Research objectives

The main objectives of this work are as follows:

- Evaluate the performance of K-Means, DBSCAN, and GMM across diverse datasets with different sizes, dimensionalities, and cluster structures.
- Compare the three algorithms in terms of cluster quality, robustness to noise, and sensitivity to parameter choices.
- Contribute to data mining research through a systematic empirical comparison of representative clustering algorithms on heterogeneous datasets.

E. Main contributions

The main contributions of this study can be summarized as follows:

- A unified benchmarking pipeline for comparing K-Means, DBSCAN, and GMM under consistent preprocessing, tuning, and evaluation settings.
- An empirical evaluation across nine benchmark datasets with varying dimensionalities, cluster geometries, overlap levels, and noise sensitivity.
- A practically oriented analysis that links clustering performance to dataset characteristics rather than reporting results in isolation.
- A discussion of computational trade-offs and preprocessing bias to support more informed algorithm selection in data mining applications.

F. Organization of the research

The remainder of this paper is structured as follows. Section II presents the background on the clustering paradigms and algorithms evaluated in this study: K-Means, DBSCAN, and Gaussian Mixture Models. Section III describes the experimental methodology, including the datasets, preprocessing pipeline, hyperparameter tuning, and evaluation protocol. Section IV reports the results and discussion across the nine benchmark datasets, combining quantitative metrics and qualitative PCA visualizations. Finally, Section V concludes the paper and highlights limitations and future work.

II. BACKGROUND: CLUSTERING PARADIGMS AND ALGORITHMS

Clustering algorithms can be interpreted through the modeling assumptions they make about what constitutes a “cluster” (e.g., compactness around a prototype, density connectivity, or an underlying probabilistic generative model). In this work, we focus on three widely used paradigms—centroid-based, density-based, and model-based clustering—because they are among the most common in data mining practice and represent distinct trade-offs in efficiency, shape flexibility, and robustness to noise [6], [7]. The specific algorithms evaluated are K-Means (centroid-based), DBSCAN (density-based), and Gaussian Mixture Models (model-based).

A. Centroid-based Clustering: K-Means

Centroid-based methods partition data into k clusters, where each cluster is represented by a prototype, usually its mean (centroid) [1], [6]. Their aim is to minimize within-cluster dispersion, making them computationally efficient and scalable to large datasets [6], [7]. However, they work best when clusters are compact, well-separated, and similar in size and density, and they are sensitive to initialization, outliers, and feature scaling [6], [7].

K-Means is the most widely used centroid-based algorithm. Given a dataset $X = \{x_1, x_2, \dots, x_n\}$, it iteratively assigns each point to the nearest centroid and updates each centroid as the mean of its assigned points [1]. Its objective is:

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2, \quad (1)$$

where μ_i is the centroid of cluster C_i . Because of this distance-based formulation, K-Means often struggles with non-convex, highly imbalanced, or overlapping clusters [6], [7].

B. Density-based Clustering: DBSCAN

Density-based clustering defines clusters as contiguous regions of high sample density separated by regions of low density, while allowing isolated samples to be labeled as noise [8], [9]. This paradigm is well suited for discovering arbitrarily shaped clusters and handling outliers, since cluster membership is based on density connectivity rather than distance to a centroid [6], [7].

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is parameterized by a neighborhood radius ε and a minimum number of points MinPts [8]. For a point x_i , the ε -neighborhood is defined as:

$$\mathcal{N}_\varepsilon(x_i) = \{x_j \in X \mid \text{dist}(x_i, x_j) \leq \varepsilon\}. \quad (2)$$

A point is a *core point* if $|\mathcal{N}_\varepsilon(x_i)| \geq \text{MinPts}$. A point is *directly density-reachable* from a core point if it lies within its ε -neighborhood. A cluster is formed by iteratively expanding from core points to all points that are density-reachable (and density-connected), while points that are not density-reachable from any core point are labeled as noise [8], [9].

DBSCAN does not require specifying the number of clusters in advance and can recover non-convex cluster shapes. However, its performance is sensitive to the choice of $(\varepsilon, \text{MinPts})$, particularly in high-dimensional data or datasets with varying densities [9].

C. Model-based Clustering: Gaussian Mixture Models (GMM)

Model-based clustering assumes the observed data are generated by a mixture of latent probability distributions, where each component corresponds to a cluster [2], [4]. Gaussian Mixture Models (GMMs) represent the data density as a weighted sum of K multivariate Gaussian components, enabling *soft* assignments (probabilistic memberships) rather than hard partitions [2], [3]. This makes GMMs effective for

TABLE I
DATASET SUMMARY (SIZE N , DIMENSIONALITY d , NUMBER OF CLASSES C).

Dataset	N	d	C
Banknote	1372	4	2
Ionosphere	351	34	2
Sonar	208	60	2
Blobs	1500	2	3
Varied	1500	2	3
Wine	178	13	3
Flame	240	2	2
Glass	214	9	6
Iris	150	4	3

overlapping clusters and for capturing elliptical cluster shapes via covariance modeling [4], [5].

A GMM defines:

$$p(x | \Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k), \quad (3)$$

where π_k are mixing weights ($\sum_k \pi_k = 1$), and (μ_k, Σ_k) are the mean and covariance of component k . Parameters are commonly estimated using the Expectation–Maximization (EM) algorithm, which alternates between computing posterior responsibilities and updating mixture parameters to improve the data log-likelihood [2], [5]. While flexible, GMMs require selecting K (and covariance form), can be initialization-sensitive, and may degrade when the true cluster structure is strongly non-Gaussian or dominated by heavy noise [4], [5].

III. METHODOLOGY

This section describes the experimental methodology adopted to compare K-Means, DBSCAN, and Gaussian Mixture Models (GMM) on nine benchmark datasets. A unified pipeline was used across all algorithms to ensure a fair comparison, covering data preprocessing, algorithm configuration, parameter tuning, and evaluation protocol.

A. Datasets

Nine labeled datasets with varying sizes, dimensionalities, and cluster complexities were selected and grouped into three categories:

- **Large:** Banknote, Ionosphere, Sonar
- **Medium:** Blobs, Varied, Wine
- **Small:** Flame, Glass, Iris

The labels were used only for external evaluation; all clustering algorithms were run in a fully unsupervised manner on the feature space of each dataset.

These datasets were selected to cover diverse cluster characteristics, including varying dimensionality, overlap, class imbalance, and noise sensitivity.

B. Common Preprocessing

To make the comparison consistent and to reduce the effect of scale and outliers, the following preprocessing steps were applied to all datasets before clustering:

- 1) **Duplicate removal:** Each dataset was scanned for duplicate rows. Exact duplicates were removed to avoid biasing distance computations and centroid updates.
- 2) **Outlier handling:** A Z-score filter was applied to all numerical features. Samples with any feature satisfying $|z| > 3$ were removed as outliers to improve cluster compactness and reduce the influence of extreme values.
- 3) **Standardization:** All remaining features were standardized using Z-score normalization

$$x' = \frac{x - \mu}{\sigma}, \quad (4)$$

where μ and σ denote the feature-wise mean and standard deviation, respectively. This step ensures that all dimensions contribute equally to Euclidean-distance-based methods and improves numerical stability for GMM.

C. Algorithm Implementations

All three clustering methods were implemented specifically for this study following their standard definitions. K-Means and DBSCAN were coded from scratch, while the GMM implementation relied on an EM routine from a numerical library wrapped in a custom clustering interface.

1) **K-Means:** K-Means partitions the standardized dataset $X = \{x_1, \dots, x_N\}$ into k clusters by minimizing the within-cluster sum of squared distances

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2, \quad (5)$$

where μ_i is the centroid of cluster C_i .

For each dataset, k was set equal to the number of ground-truth classes.

Notation. We use k to denote the number of clusters in K-Means, while K denotes the number of mixture components in GMM.

The algorithm used Euclidean distance for assignment, random initialization of centroids with a fixed random seed for reproducibility, and terminated when either the total centroid displacement between iterations fell below 10^{-4} or a maximum of 1000 iterations was reached.

2) **DBSCAN:** DBSCAN was applied in the standardized feature space using Euclidean distance. The algorithm groups points that lie in dense regions and labels isolated points as noise [8], [9]. It is controlled by a neighborhood radius ε and a minimum number of points MinPts.

Parameter selection. To avoid using ground-truth labels during tuning, ε was selected using the k -distance curve: for each dataset we set $k_{nn} = \text{MinPts}$, computed the sorted distance to the k_{nn} -th nearest neighbor,

and identified a knee region corresponding to a transition from dense points to sparser points. We then performed a small local search around the knee region, testing candidate ε values in a narrow interval.

MinPts was selected relative to dimensionality following common practice, using a small tested set around $2d$ (where

TABLE II
FINAL HYPERPARAMETERS USED FOR EACH DATASET.

Dataset	K-Means k	DBSCAN (ϵ , MinPts)	GMM K / cov.
Banknote	2	(0.2, 10)	2 / full
Ionosphere	2	(0.4, 3)	2 / full
Sonar	2	(0.6, 3)	2 / full
Blobs	3	(0.2, 3)	3 / full
Varied	3	(0.2, 8)	3 / full
Wine	3	(0.4, 5)	3 / full
Flame	2	(0.2, 3)	2 / full
Glass	6	(0.2, 5)	6 / full
Iris	3	(0.2, 3)	3 / full

d is the number of features after preprocessing). Among candidate pairs (ϵ , MinPts), the final configuration was selected using an *internal* clustering criterion: the silhouette score computed on *non-noise* points only. This choice maintains an unsupervised tuning procedure while still producing stable clusterings.

The final hyperparameters used for each dataset are summarized in Table II.

3) *Gaussian Mixture Models (GMM)*: GMM models the data distribution as a mixture of K multivariate Gaussian components

$$p(x | \Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k), \quad (6)$$

where π_k are the mixing weights, and μ_k and Σ_k denote the mean vector and covariance matrix of component k .

The number of components K was set equal to the number of classes in each dataset. Parameters $\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ were estimated using the Expectation–Maximization (EM) algorithm initialized with K-Means centroids. The EM iterations were stopped when the absolute change in log-likelihood between consecutive iterations fell below 10^{-4} or when a maximum number of iterations was reached.

D. Hyperparameter Settings

Table II reports the final hyperparameter settings used for each dataset after the tuning procedure described above. This summary supports reproducibility and clarifies the configurations underlying the results.

Table II reports only the dataset-dependent hyperparameters, while algorithmic settings common to all datasets are described in Section III-C.

E. Experimental Setup

To improve robustness and account for initialization effects, each algorithm was executed for 10 independent runs under the final hyperparameter configuration reported in Table II. K-Means used different random initializations across runs, while GMM was initialized from K-Means centroids under the same experimental protocol. The values reported in Section IV represent mean performance across runs. In our experiments, the observed variability was small, with standard deviation remaining negligible for the main evaluation metrics; therefore,

mean values are reported in the main comparison table for readability.

F. Evaluation Protocol

Although the clustering algorithms do not use labels during training, no ground-truth labels were used for parameter tuning or clustering; labels were used only after clustering to compute external metrics. **Benchmarking note.** For standardized benchmarking, the number of clusters/components for K-Means and GMM was set to the known number of classes (C), while DBSCAN was tuned fully unsupervised.

Setting the number of clusters k or components K equal to the ground-truth class count C allows for an “upper-bound” performance assessment. This evaluates how effectively each algorithm’s geometric assumptions align with the true underlying data structure in an ideal-case parameterization.

For each dataset and algorithm, the following metrics were computed after aligning cluster IDs to class labels using majority voting:

- **Purity**: measures the proportion of the dominant class in each cluster and averages this over all clusters.
- **Accuracy**: classification accuracy obtained after mapping clusters to classes.
- **Macro-Precision**: average of class-wise precision values.
- **Macro-Recall**: average of class-wise recall values.
- **Macro F1-Score**: harmonic mean of macro-precision and macro-recall.

All metrics were computed on the full dataset (no train–test split), since the goal is to assess how well the discovered clusters recover the known structure.

Treatment of DBSCAN noise points. DBSCAN may label some samples as noise rather than assigning them to any cluster. For external, label-based metrics (purity, accuracy, macro-precision, macro-recall, and macro F1-score), noise points were treated as incorrect assignments by assigning them to a dedicated “noise” label that is not mapped to any ground-truth class.

We additionally computed silhouette scores in the standardized feature space. For DBSCAN, silhouette was computed on non-noise points only; when fewer than two clusters were formed (or when computation was ill-defined), the score is reported as NaN.

G. Visualization

To provide an interpretable view of the clustering structure, Principal Component Analysis (PCA) was applied to each preprocessed dataset. The first two principal components were used to generate 2D scatter plots showing data points colored by their cluster label and, where applicable, annotated with centroids or component means. These visualizations, reported in Section IV, support qualitative comparison of cluster separability and shape alongside the quantitative results.

IV. RESULTS AND DISCUSSION

Tables III–V summarize the evaluation metrics for DBSCAN, GMM, and K-Means across the benchmark datasets

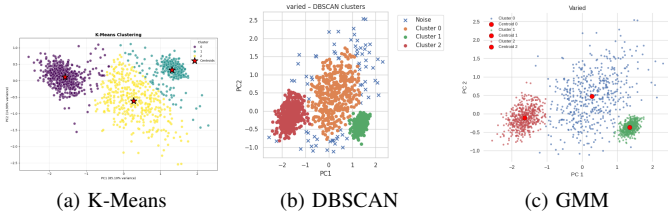


Fig. 1. Representative PCA visualization (example: Varied dataset) comparing K-Means, DBSCAN, and GMM.

TABLE III

EVALUATION METRICS FOR DBSCAN CLUSTERING (MEAN $\pm$ STANDARD DEVIATION OVER MULTIPLE RUNS).

Dataset	Purity	Accuracy	Prec.	Rec.	F1	Silh.
Banknote	68.86 $\pm$ 0	68.86 $\pm$ 0	82.41 $\pm$ 0	63.48 $\pm$ 0	60.56 $\pm$ 0	0.46 $\pm$ 0
Ionosphere	83.26 $\pm$ 0	83.26 $\pm$ 0	83.68 $\pm$ 0	78.09 $\pm$ 0	79.84 $\pm$ 0	0.59$\pm$0
Sonar	63.04 $\pm$ 0	63.04 $\pm$ 0	63.21 $\pm$ 0	63.19 $\pm$ 0	63.04 $\pm$ 0	0.53$\pm$0
Blobs	100.00$\pm$0	100.0$\pm$0	100.00$\pm$0	100.0$\pm$0	100.00$\pm$0	0.81$\pm$0
Varied	98.67$\pm$0	98.67$\pm$0	98.73$\pm$0	98.53$\pm$0	98.62$\pm$0	0.66$\pm$0
Wine	97.46$\pm$0	97.46$\pm$0	97.87$\pm$0	97.73$\pm$0	97.73$\pm$0	0.63$\pm$0
Flame	100.00$\pm$0	100.00$\pm$0	100.00$\pm$0	100.00$\pm$0	100.00$\pm$0	0.31 $\pm$ 0
Glass	55.22 $\pm$ 0	55.22 $\pm$ 0	57.27 $\pm$ 0	54.17 $\pm$ 0	47.97 $\pm$ 0	0.76$\pm$0
Iris	94.90$\pm$0	94.90$\pm$0	95.10$\pm$0	94.44$\pm$0	94.32$\pm$0	0.67$\pm$0

TABLE IV

EVALUATION METRICS FOR GMM CLUSTERING (MEAN $\pm$ STANDARD DEVIATION OVER MULTIPLE RUNS).

Dataset	Pur.	Acc.	Pre.	Rec.	F1	Silh.
Banknote	55.51 $\pm$ 0	55.51 $\pm$ 0	27.75 $\pm$ 0	50 $\pm$ 0	35.69 $\pm$ 0	0.29 $\pm$ 0.04
Ionosphere	72.57 $\pm$ 8.64	72.57 $\pm$ 8.64	78.20 $\pm$ 8.52	65.31 $\pm$ 13.32	62.08 $\pm$ 15.73	0.22 $\pm$ 0.08
Sonar	53.78 $\pm$ 0.36	53.78 $\pm$ 0.36	44.28 $\pm$ 12.38	51.24 $\pm$ 1.49	42.18 $\pm$ 6.67	0.02 $\pm$ 0.02
Blobs	100$\pm$0	100$\pm$0	100$\pm$0	100$\pm$0	100$\pm$0	0.81$\pm$0
Varied	98.87$\pm$0	98.87$\pm$0	98.87$\pm$0	98.87$\pm$0	98.86$\pm$0	0.60$\pm$0
Wine	57.12 $\pm$ 9.66	57.12 $\pm$ 9.66	49.41 $\pm$ 19.07	58.09 $\pm$ 11.99	49.38 $\pm$ 14.40	0.10 $\pm$ 0.01
Flame	75.40 $\pm$ 2.14	75.40 $\pm$ 2.14	79.53 $\pm$ 1.28	80.68 $\pm$ 1.80	75.36 $\pm$ 2.11	0.35 $\pm$ 0.01
Glass	49.95 $\pm$ 2.40	49.95 $\pm$ 2.40	31.69 $\pm$ 5.21	38.49 $\pm$ 5.55	32.92 $\pm$ 4.34	0.18 $\pm$ 0.03
Iris	79.33 $\pm$ 10.51	79.33 $\pm$ 10.51	76.64 $\pm$ 18.06	79.47 $\pm$ 10.44	75.41 $\pm$ 14.91	0.33 $\pm$ 0.03

and highlight clear performance trends across heterogeneous data conditions. DBSCAN achieves the strongest overall robustness on datasets with clear density separation and potential noise, reaching perfect or near-perfect performance on *Blobs*, *Flame*, and strong results on *Varied*, *Wine*, and *Iris*. In contrast, K-Means performs best when clusters are compact and well separated (e.g., *Wine* and *Varied*), but degrades on datasets with overlap or irregular structure (e.g., *Glass* and *Banknote*). GMM is competitive on datasets that approximate Gaussian or elliptical components (e.g., *Ionosphere* and *Varied*), but its performance declines when the data distribution is strongly non-Gaussian or noisy (e.g., *Glass* and *Sonar*).

The representative PCA visualization in Fig. 1 qualitatively supports these observations by illustrating how centroid-based partitions may mis-handle overlap, while density connectivity and covariance modeling can better reflect complex structure depending on the dataset.

A. Key Findings

Across the benchmark datasets, DBSCAN is generally the most robust method for non-convex and noisy cluster structures, benefiting from density connectivity and explicit

TABLE V
EVALUATION METRICS FOR K-MEANS CLUSTERING (MEAN $\pm$ STANDARD DEVIATION OVER MULTIPLE RUNS).

Dataset	Purity	Accuracy	Prec.	Rec.	F1	Silh.
Banknote	55.4 $\pm$ 0.0	55.2 $\pm$ 0.03	49.8 $\pm$ 0.03	57.5 $\pm$ 0.06	53.3 $\pm$ 0.04	0.34 $\pm$ 0.0
Ionosphere	70.3 $\pm$ 1.3	70.3 $\pm$ 1.3	59.4 $\pm$ 9.8	68.3 $\pm$ 16.2	60.4 $\pm$ 13.9	0.27 $\pm$ 0.01
Sonar	55.4 $\pm$ 2.5	55.1 $\pm$ 2.8	63.2 $\pm$ 3.2	39.1 $\pm$ 11.5	47.2 $\pm$ 8.7	0.18 $\pm$ 0.04
Blobs	88.7 $\pm$ 15.8	83.3 $\pm$ 23.2	83.0 $\pm$ 23.7	83.3 $\pm$ 23.2	81.4 $\pm$ 25.9	0.74$\pm$0.09
Varied	93.1$\pm$0.0	93.1$\pm$0.0	93.8$\pm$0.0	93.1$\pm$0.0	92.9$\pm$0.0	0.63$\pm$0.0
Wine	94.0$\pm$7.9	93.8$\pm$8.8	94.4$\pm$8.4	93.8$\pm$8.8	93.3$\pm$10.7	0.28 $\pm$ 0.02
Flame	90.9 $\pm$ 7.8	67.3 $\pm$ 4.1	94.0 $\pm$ 4.0	67.3 $\pm$ 4.1	77.2 $\pm$ 4.0	0.40 $\pm$ 0.02
Glass	41.6 $\pm$ 5.1	29.8 $\pm$ 3.0	11.8 $\pm$ 1.0	29.8 $\pm$ 3.0	16.9 $\pm$ 1.5	0.38 $\pm$ 0.07

TABLE VI

SUMMARY OF THE COMPUTATIONAL CHARACTERISTICS OF THE EVALUATED CLUSTERING ALGORITHMS.

Algorithm	Typical complexity	Practical observation
K-Means	$O(n \cdot k \cdot d \cdot i)$	Fastest in most settings
DBSCAN	Depends on neighborhood queries	Sensitive to ϵ and data density
GMM	$O(i \cdot n \cdot k \cdot d^2)$	Slower on high-dimensional data

noise labeling [8], [9]. K-Means remains effective on compact, well-separated clusters because of its centroid-based objective [6], [7]. GMM performs well when clusters are elliptical or partially overlapping, as its covariance modeling and soft assignments can better capture such patterns [2], [4], [5]. These trends are consistent with the assumptions and limitations of each method discussed in Section II.

Our results on non-convex structures, such as *Flame* and *Blobs* where DBSCAN achieved near-perfect F1-scores, agree with Murugesan et al. [5], who reported that density-based methods outperform centroid-based models on non-spherical clusters. At the same time, our results show that GMM remains competitive on the *Varied* dataset, suggesting that probabilistic covariance modeling can also perform well on overlapping or elliptical structures, a point less emphasized in broader clustering surveys [7].

Failure cases: DBSCAN degrades on overlapping or varying-density datasets (e.g., *Banknote* and *Sonar*), where a single global ϵ is insufficient. GMM struggles when data deviate strongly from Gaussian components or contain heavy noise, while K-Means performs poorly on non-convex or imbalanced clusters due to its spherical-cluster bias.

These findings directly address the study gap identified in Section I. By evaluating centroid-based, density-based, and model-based clustering under a unified preprocessing and evaluation pipeline across nine heterogeneous datasets, the study shows how algorithmic assumptions lead to different empirical behaviors under varied data conditions. This helps bridge the gap between theoretical expectations and practical algorithm selection in applied data mining.

B. Complexity and Preprocessing Bias

Table VI summarizes the main computational characteristics of the evaluated clustering algorithms, complementing the empirical results with a concise comparison of their expected

computational behavior. While runtime was briefly noted in the results analysis, a systematic review of algorithmic complexity reveals significant trade-offs. K-Means remains the most efficient at $O(n \cdot k \cdot d \cdot i)$, making it the standard baseline for large-scale data [6], [7]. In contrast, GMM’s iterative EM updates and covariance modeling $O(i \cdot n \cdot k \cdot d^2)$ result in higher computational costs, particularly on high-dimensional datasets like Sonar [2], [5].

Furthermore, the impact of preprocessing must be considered. While the Z-score filtering ($|z| > 3$) implemented in our pipeline improved cluster compactness for K-Means [6], it potentially simplifies the task for DBSCAN by removing the extreme outliers it is inherently designed to label as noise [8], [9]. This suggests that DBSCAN’s performance in this study represents its efficacy on refined datasets, and its robustness on raw, heavy-noise data remains an area for further investigation.

C. Practical Implications

From a practical perspective, the results suggest that K-Means is most suitable when clusters are expected to be compact, well separated, and computational efficiency is a priority. DBSCAN is more appropriate when the data may contain noise, outliers, or irregularly shaped clusters, although careful parameter tuning remains essential. GMM is preferable when the underlying structure is expected to involve overlap or approximately elliptical cluster distributions, particularly in applications where soft probabilistic assignments are meaningful. These observations provide a useful reference for selecting clustering methods in real-world data mining tasks where the true structure of the data is not known in advance.

V. CONCLUSION

This study compared K-Means, DBSCAN, and Gaussian Mixture Models on nine benchmark datasets using a unified preprocessing and evaluation pipeline. The experiments showed that clustering performance is highly dependent on data characteristics such as cluster shape, overlap, and noise. K-Means performed well on compact, well-separated clusters but degraded on irregular or imbalanced structures. GMM was effective when clusters followed approximately Gaussian, elliptical distributions, while DBSCAN was most robust to noise and capable of identifying non-convex clusters, albeit with high sensitivity to parameter selection.

Overall, the results confirm that no single clustering algorithm is universally optimal. Instead, the choice of method should be driven by prior knowledge of the underlying data distribution and the application requirements. In practice, K-Means is suitable as a fast baseline, GMM is preferable for overlapping or elliptical clusters, and DBSCAN is advantageous when dealing with noisy data and complex cluster shapes.

Limitations and future work: The study relies on external labels for evaluation only, and the reported results remain sensitive to preprocessing choices and algorithm parameter selection. In addition, K-Means and GMM were evaluated

using the known class count as an upper-bound parameterization for standardized benchmarking, which does not fully reflect real unsupervised deployment where the number of clusters is unknown. PCA projections also provide only a qualitative low-dimensional view and may obscure structure in higher-dimensional data. Future work should therefore investigate fully unsupervised cluster-number selection, compare performance with and without outlier filtering, and include a more systematic empirical analysis of computational cost across datasets.

REFERENCES

- [1] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. New York, NY, USA: Wiley, 1990.
- [2] S. Sarkar *et al.*, “Gaussian mixture modeling and model-based clustering under measurement inconsistency,” *Advances in Data Analysis and Classification*, vol. 14, no. 5, pp. 1–35, 2020.
- [3] B. Zhao *et al.*, “Learning semi-supervised Gaussian mixture models for generalized category discovery,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.
- [4] A. Lind, S. Wu, and A. Hadachi, “Application of Gaussian mixtures in a multimodal Kalman filter to estimate the state of a nonlinearly moving system,” *Sensors*, vol. 23, no. 7, Art. no. 3603, 2023.
- [5] N. Murugesan, I. Cho, and C. Tortora, “Benchmarking in cluster analysis: A study on spectral clustering, DBSCAN, and k-means,” *Advances in Data Analysis and Classification*, 2021.
- [6] A. K. Jain, M. N. Murty, and P. J. Flynn, “Data clustering: A review,” *ACM Comput. Surveys*, vol. 31, no. 3, pp. 264–323, 1999.
- [7] R. Xu and D. C. Wunsch, “Survey of clustering algorithms,” *IEEE Trans. Neural Netw.*, vol. 16, no. 3, pp. 645–678, 2005.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining (KDD)*, 1996, pp. 226–231.
- [9] E. Schubert, J. Sander, M. Ester, H.-P. Kriegel, and X. Xu, “DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN,” *ACM Trans. Database Syst.*, vol. 42, no. 3, pp. 1–21, 2017.
- [10] M. Halkidi, Y. Batistakis, and M. Vazirgiannis, “On clustering validation techniques,” *J. Intell. Inf. Syst.*, vol. 17, pp. 107–145, 2001.
- [11] D. Birant and A. Kut, “ST-DBSCAN: An algorithm for clustering spatial-temporal data,” *Data Knowl. Eng.*, vol. 60, no. 1, pp. 208–221, 2007.
- [12] M. E. Celebi, H. A. Kingravi, and P. A. Vela, “A comparative study of efficient initialization methods for the k-means clustering algorithm,” *Expert Syst. Appl.*, vol. 40, no. 1, pp. 240–257, 2013.